

LINQ ile veriyi sorgulamayı öğren

Listeyi değiştirmeden filtrele, dönüştür, sırala ve özetle. Önce küçük çalışan örnekler; sonra senin tamamlayacağın iskeletler.

Yaklaşık	Odak	Çıktı
150–210 dk	Metot sözdizimi	Mini ürün raporu

Bugünün çalışma kuralı

Çalışan örneği aynen çalıştır. Sonra kapatıp iskeleti kendin tamamla. Öğrenci kodu görülmeden görevlerin tam çözümü verilmez.

Bitiş ölçütü

- Where, Select ve lambda ifadesini kendi cümlemlerle açıklayabilmek
- Any / All / FirstOrDefault farkını doğru seçebilmek
- Bir ürün listesini filtreleyip sıralayarak özet değer üretebilmek
- Mini uygulamayı en az üç senaryoyla çalıştırmak

1. LINQ nedir?

LINQ (Language Integrated Query), koleksiyonlar üzerinde “hangi veriyi istiyorum?” diye sorgu yazmamızı sağlar. Uzun döngüler yerine niyeti daha açık bir zincir kurar.

Zihinsel model

Kaynak İşlem Sonuç

Liste filtreleme yeni sonuç listesi. Kaynak liste genellikle yerinde değişmez.

Klasik yaklaşım: foreach + if

```
List<int> sayilar = new() { 5, 12, 18, 3 };
List<int> buyukler = new();

foreach (int sayi in sayilar)
{
    if (sayi > 10)
        buyukler.Add(sayi);
}

Console.WriteLine(string.Join(", ", buyukler)); // 12, 18
```

Aynı amaç: Where

```
List<int> sayilar = new() { 5, 12, 18, 3 };
List<int> buyukler = sayilar
    .Where(sayi => sayi > 10)
    .ToList();

Console.WriteLine(string.Join(", ", buyukler)); // 12, 18
```

Where, koşulu sağlayan elemanları seçer. ToList(), sorgu sonucunu gerçek bir listeye dönüştürür.

Dikkat

Where kaynak listeyi silmez veya düzenlemez; eşleşenlerden yeni bir sonuç üretir.

2. Lambda: küçük bir koşul cümlesi

Şu ifadeyi soldan sağa oku:

```
sayi => sayi > 10
```

sayi	=>	sayi > 10
Listedeki sıradaki eleman	"şunu yap"	Kontrol edilecek koşul

Görev 1 — Yalnızca çift sayılar

Aşağıdaki iskeleti tamamla. Tam çözüm özellikle bırakılmamıştır.

```
List<int> sayilar = new() { 3, 8, 11, 14, 20, 25 };

// 1) Where kullanarak çift sayıları filtrele.
// İpucu: Bir sayı 2'ye tam bölünüyorsa kalan 0'dır.
List<int> ciftSayilar = sayilar
    .Where(sayi => /* koşul buraya */)
    .ToList();

// 2) Sonucu tek satırda yazdır.
Console.WriteLine(/* burayı tamamla */);
```

Beklenen çıktı

```
8, 14, 20
```

Kendini kontrol et

Kaynak listedeki 3 ve 25 hâlâ duruyor mu? Sonuç listesinde yalnızca üç çift sayı var mı?

Kodu yazıp çalıştırdıktan sonra gönder. Önce doğru yaptığın yerleri, sonra varsa hataları ve nedenlerini birlikte inceleyeceğiz.

3. Select: her elemanı dönüştür

Where seçer. Select dönüştürür. Sonuçtaki eleman sayısı çoğunlukla kaynakla aynıdır; fakat elemanın değeri veya tipi değişebilir.

Küçük çalışan örnek

```
List<int> sayilar = new() { 2, 3, 4 };
List<int> kareler = sayilar
    .Select(sayi => sayi * sayi)
    .ToList();

Console.WriteLine(string.Join(", ", kareler)); // 4, 9, 16
```

Soru

Where ile Select yer değiştirirse sonuç her zaman aynı olur mu? Hayır. Zinciri soldan sağa, adım adım düşün.

Görev 2 — Etiket üret

Sayıları "Sayı: 2" biçiminde metne dönüştür.

```
List<int> sayilar = new() { 2, 3, 4 };

List<string> etiketler = sayilar
    .Select(sayi => /* string üret */)
    .ToList();

foreach (string etiket in etiketler)
{
    Console.WriteLine(etiket);
}
```

Beklenen biçim:

```
Sayı: 2
Sayı: 3
Sayı: 4
```

4. Bulmak ve doğrulamak

Bu üç metot benzer görünür; sordukları soru farklıdır.

Metot	Sorduğu soru	Sonuç
FirstOrDefault	İlk eşleşen hangisi?	Eleman veya varsayılan değer
Any	En az bir eşleşme var mı?	bool
All	Hepsi koşulu sağlıyor mu?	bool

Küçük çalışan örnek

```
List<int> sayilar = new() { 4, 9, 12 };

int ilkBuyuk = sayilar.FirstOrDefault(s => s > 10); // 12
bool tekVarMi = sayilar.Any(s => s % 2 != 0);      // true
bool hepsiPozitifMi = sayilar.All(s => s > 0);     // true
```

Önemli ayrıntı

int için eşleşme yoksa FirstOrDefault sonucu 0 olur. 0 listedeki geçerli bir değer olabileceği için kullanım amacını düşün.

Görev 3 — Doğru metodu seç

```
List<int> notlar = new() { 55, 70, 92, 40 };

// En az bir 90+ not var mı?
bool yuksekNotVarMi = /* ... */;

// Bütün notlar 0-100 aralığında mı?
bool notlarGecerliMi = /* ... */;

// İlk başarısız notu bul (başarı sınırı 50).
int ilkBasarisiz = /* ... */;
```

5. Sırala ve özetle

Sıralama metotları yeni bir sıra üretir. Toplama metotları ise koleksiyondan tek bir özet değeri çıkarır.

Küçük çalışan örnek

```
List<int> puanlar = new() { 70, 40, 90, 60 };

List<int> azalan = puanlar
    .OrderByDescending(puan => puan)
    .ToList();

int adet = puanlar.Count();
int toplam = puanlar.Sum();
double ortalama = puanlar.Average();
int enDusuk = puanlar.Min();
int enYuksek = puanlar.Max();
```

OrderBy	Küçükten büyüğe / A'dan Z'ye
OrderByDescending	Büyükten küçüğe / Z'den A'ya
Count · Sum · Average	Adet · toplam · ortalama
Min · Max	En küçük · en büyük

Görev 4 — Puan raporu

```
List<int> puanlar = new() { 70, 40, 90, 60 };

// 1) Puanları küçükten büyüğe sırala.
// 2) 50 ve üzerindeki puanların sayısını bul.
// 3) Tüm puanların ortalamasını yazdır.
// Her sonucu anlamlı bir etiketle ekrana yazdır.
```

İpucu

Önce Where ile filtreleyip sonra Count kullanabilirsin. Zinciri soldan sağa oku.

6. Nesnelerle LINQ ve GroupBy

LINQ yalnızca sayılarla çalışmaz. Nesnelerin property'lerine göre de filtreleme, sıralama ve gruptama yapabiliriz.

Başlangıç modeli

```
public class Product
{
    public string Name { get; set; } = "";
    public string Category { get; set; } = "";
    public decimal Price { get; set; }
    public int Stock { get; set; }
}
```

Küçük çalışan örnek: yalnızca isimleri seç

```
List<string> adlar = products
    .Select(urun => urun.Name)
    .ToList();
```

GroupBy, aynı anahtara sahip elemanları gruplar. Örneğin kategoriye göre gruptama:

```
var gruplar = products.GroupBy(urun => urun.Category);

foreach (var grup in gruplar)
{
    Console.WriteLine($"Kategori: {grup.Key}");
    Console.WriteLine($"Ürün sayısı: {grup.Count()}");
}
```

Buradaki var ne?

Derleyici sağ taraftaki LINQ sonucundan tipi çıkarır. "Tip yok" anlamına gelmez.

Mini kontrol

- grup.Key ortak kategori adı
- grup o kategoriye ait ürünlerin koleksiyonu
- grup.Count() grubun içindeki ürün sayısı

7. Mini uygulama — Ürün raporu

Şimdi parçaları birleştir. Aşağıdaki iskelet bilinçli olarak eksiktir; tam çözüm öğrenci kodu görülmeden verilmez.

Görev

- Fiyatı 500 TL ve üzerindeki ürünleri filtrele.
- Filtrelenenleri fiyata göre pahalıdan ucuza sırala.
- Stokta en az bir ürün olup olmadığını kontrol et.
- Ortalama fiyatı hesapla.
- Ürünleri kategoriye göre gruba ve her grubun adedini yazdır.

```
List<Product> products = new()
{
    new Product { Name = "Klavye", Category = "Çevre Birimi", Price = 850, Stock = 5 },
    new Product { Name = "Mouse", Category = "Çevre Birimi", Price = 450, Stock = 0 },
    new Product { Name = "Monitör", Category = "Ekran", Price = 6200, Stock = 3 },
    new Product { Name = "Webcam", Category = "Kamera", Price = 1200, Stock = 2 }
};

// 1) 500 TL ve üzerini Where ile filtrele.
var filtrelenenler = /* ... */;

// 2) Sonucu OrderByDescending ile sırala.
var siralananlar = /* ... */;

// 3) Stok kontrolü için Any kullan.
bool stokVarMi = /* ... */;

// 4) Average ile ortalama fiyatı bul.
decimal ortalamaFiyat = /* ... */;

// 5) Category alanına göre GroupBy yap.
var kategoriGruplari = /* ... */;

// 6) Sonuçları foreach ile anlamlı etiketlerle yazdır.
```

Test senaryoları

Senaryo	Kontrol
Normal liste	Filtre ve sıralama doğru mu?
Tüm stoklar 0	Any sonucu false mu?
Boş liste	Average çağrısından önce kontrol var mı?

8. Kapanış ve mülakat notları

Kısa cevaplarla hatırlanacaklar

LINQ nedir?

Koleksiyonlar üzerinde filtreleme, dönüştürme, sıralama ve özetleme sorgularını C# içinde yazmayı sağlar.

Where ve Select farkı

Where eleman seçer; Select her elemanı başka bir değere veya tipe dönüştürür.

Any ve All farkı

Any için bir eşleşme yeterlidir. All için bütün elemanlar koşulu sağlamalıdır.

Bugün tamamlandı sayılması için

- Dört kısa görevin en az üçünü yardımsız tamamla.
- Mini ürün raporunu çalıştır ve üç test senaryosunu dene.
- Bir LINQ zincirini soldan sağa sözlü olarak açıkla.
- Kodu gönder; hataları nedenleriyle birlikte düzeltelim.

Proje ve Git

```
cd "C:\Users\Administrator\.NetEgitim\DotNetIlkHafta"
mkdir Gun5-LINQ
cd Gun5-LINQ
dotnet new console
code .
```

```
# Çalışma bittikten sonra repository kökünden:
cd "C:\Users\Administrator\.NetEgitim"
git add .
git commit -m "Gün 5 LINQ alıştırımları tamamlandı"
git push
```

Sonraki adım

Önce Görev 1'i tamamla ve kodunu gönder. Tam çözüm yerine kodunun güçlü yanlarını ve geliştirilmesi gereken noktaları birlikte inceleyeceğiz.